

CHAPTER 13 | EXPERIMENTAL SPECIFICATION v1.0

Build Agents That Get Cheaper

Designing Compounding Agent Systems Whose Marginal Reasoning Cost Falls as Validated Knowledge Accumulates

PURPOSE

Chapter 13 tests the cumulative claim behind the book: can an agent become measurably cheaper to operate as it gains validated experience, rather than merely saving tokens on isolated repeated tasks? The experiment measures cost and correctness across successive workload waves while a governed capability library grows, survives compatible reuse, and is selectively repaired when the environment changes.

The target is not to prove that every request eventually becomes deterministic. Some work should remain on the reasoning path. The target is to determine whether validated knowledge can accumulate faster than maintenance burden, producing a declining marginal LLM cost per request without degrading correctness.

This specification freezes the experimental design before implementation and before any canonical real-model benchmark. No benchmark result, savings percentage, or winning architecture is assumed in advance.

1. Research Question

Can a persistent agent reduce its marginal LLM calls, tokens, and measured provider cost across repeated workload waves by accumulating validated reusable capabilities, while preserving correctness and safely returning to reasoning when novelty or drift invalidates prior knowledge?

CHAPTER PRINCIPLE

A Smart Agent should not merely be cheaper than a baseline once. It should have a defensible mechanism by which validated experience compounds: new reasoning becomes reusable knowledge, reusable knowledge suppresses future reasoning, and invalid knowledge is repaired or retired rather than blindly replayed.

The key dependent variable is the cost curve over time. Chapter 13 must show whether the marginal intelligence cost of later requests actually falls as the system learns - and exactly why it falls or fails to fall.

2. Experimental Scenario

The benchmark again uses HarborPoint Distribution Group. Instead of a single isolated optimization decision, the company experiences a sequence of operational waves. Familiar work returns, new task families appear, related variants share reusable primitives, and selected business contracts drift.

The environment is controlled and synthetic so that ground truth, novelty, recurrence, compatibility, drift, capability acquisition, validation, and invalidation can be audited independently of the model. Real-model runs measure actual provider usage; deterministic benchmark accounting records capability-side work separately.

3. The Eight Operational Families

Family	Operational work	Learning behavior
A. Inventory	Reorder and stock-position decisions	Stable core rule; repeated heavily.
B. Shipment SLA	Late-shipment and escalation decisions	Stable; high reuse potential.
C. Invoice	Escalation and collection actions	Stable at first; contract changes later.
D. Supplier	Supplier scoring and review thresholds	Reusable scoring primitive; later policy drift.
E. Customer	Account action and service decisions	Stable but includes reasoning-required exceptions.
F. Order	Priority and fulfillment decisions	Recurring; SLA changes in a later wave.
G. Returns	Return eligibility and disposition	Introduced after the system has already learned related policy patterns.
H. Capacity	Warehouse capacity and overflow action	Introduced late; tests whether novelty still triggers fresh reasoning.

Families G and H are deliberately introduced after the initial learning wave. They prevent the benchmark from becoming a closed-world replay exercise. A system that has become cheaper must still recognize genuinely new work and spend reasoning when necessary.

4. Architectures Under Test

Architecture 1 - Stateless Reason. Every request is handled by the LLM. No capability memory persists between requests. This establishes the flat-cost reasoning baseline.

Architecture 2 - Episodic Reuse. The agent may create deterministic capabilities inside a workload wave, but its reusable registry is reset at the beginning of every new wave. It can exploit local repetition but cannot compound learning across time. This isolates the value of persistence.

Architecture 3 - Compounding Smart Agent. A governed capability registry persists across waves. Successful reasoning may be promoted only after validation. Compatible capabilities are reused deterministically. New families and reasoning-required exceptions return to the LLM. Drift triggers selective invalidation and relearning rather than global forgetting.

CRITICAL DISTINCTION

Chapter 13 is not another one-shot cache benchmark. The Compounding architecture must carry validated knowledge forward across explicit wave boundaries. If the registry is rebuilt from scratch each wave, the experiment does not test compounding learning.

5. What Counts as Compounding Knowledge?

A reusable artifact must be more than a stored answer. Each promoted capability must carry enough metadata for deterministic compatibility checks and independent audit.

```
ValidatedCapability
capability_id
family
operation
contract_version
input_schema / output_schema
implementation_hash
dependency_fingerprint
policy_fingerprint
environment_fingerprint
validation_suite_id
validation_status
provenance
acquired_at_sequence
last_validated_sequence
reuse_count
invalidation_reason
```

A capability contributes to compounding only when a later request can execute it with new inputs without another LLM call and the compatibility gate can justify that reuse from current observable evidence.

6. Learning State Machine

```
UNKNOWN
-> LLM reasoning
-> candidate capability
-> deterministic validation
fail -> REASONING_ONLY / RETRY
pass -> PROMOTED

PROMOTED
-> compatible request -> DETERMINISTIC_REUSE
-> reasoning-required exception -> LLM reasoning
-> incompatible contract/environment -> INVALIDATE

INVALIDATE
-> fresh LLM reasoning
-> validate replacement
pass -> RELEARNED
fail -> REASONING_ONLY
```

The state machine must be software-owned. The LLM may propose an answer or capability candidate, but it must not decide whether its own previous capability is compatible. Compatibility, promotion, invalidation, and registry mutation must be deterministic and auditable.

7. Canonical Workload Design

The canonical benchmark will contain exactly 240 requests per architecture, divided into four ordered waves of 60 requests. The same authoritative workload stream must be used for all architectures.

Wave	Requests	Purpose
Wave 1 - Learn	1-60	Six initial families appear. Repetition exists, but the persistent registry begins empty.
Wave 2 - Reuse	61-120	Compatible familiar work dominates. Returns is introduced. Tests whether prior learning lowers marginal cost.
Wave 3 - Drift	121-180	Selected invoice, supplier, and order contracts change. Familiar stable families remain compatible.
Wave 4 - Mature + Novel	181-240	Stable learned work continues; Capacity is introduced; reasoning-required exceptions remain present.

Exact request IDs, family interleaving, contract versions, drift points, exception flags, and new-family introduction points must be generated by code and frozen before the canonical real benchmark. The workload hash becomes part of the evidence manifest.

8. Anti-Leakage Rules

The benchmark must make later requests unknowable to the decision path. The Compounding agent may use only state accumulated from requests already processed plus the current request and its current public operational contract.

Prohibited information	Reason
Future request counts or family mix	Would let the agent optimize using benchmark hindsight.
Future drift points or future contract versions	Would turn selective invalidation into foreknowledge.
Requests remaining in the canonical benchmark	Would leak the artificial benchmark horizon.
Hidden ground truth	Ground truth belongs only to scoring/audit.
Other architectures' outcomes	Would contaminate independent execution.
Future capability success/failure	Promotion must use evidence available at the time.

INFORMATION BOUNDARY

The model and routing policy may know the current family, current request, current public contract, current registry, and past observed evidence. They may not know the future workload.

9. Controlled Drift

Drift is necessary because a declining cost curve is meaningless if the benchmark assumes that learned knowledge never expires. The canonical workload therefore changes selected contracts in Wave 3 while leaving other families stable.

The exact changed families and contract versions must be frozen before the real benchmark. A compatible stable capability should continue to be reused. An affected capability must be invalidated before deterministic execution under the new contract. Global registry reset is prohibited for the Compounding architecture because it would hide whether selective maintenance works.

10. Cost and Learning Metrics

Chapter 13 must report both whole-run totals and per-wave measurements. Whole-run savings alone cannot establish that the agent is getting cheaper over time.

Metric	Definition
LLM calls	Actual provider calls, total and by wave.
Input/output/total tokens	Actual provider usage, total and by wave.
Measured provider cost	Provider cost calculated from frozen pricing constants.
Deterministic reuses	Requests completed from compatible validated capability without LLM reasoning.
Initial acquisitions	First successful capability promotions.
Relearning calls	Fresh reasoning caused by invalidated prior capability.
Reasoning-required calls	Requests intentionally kept on the model path.
Novel-family calls	Calls caused by genuinely unseen task families.
Stale-reuse errors	Incorrect deterministic results caused by incompatible reuse.
Registry size	Active validated capabilities at each wave boundary.
Marginal LLM calls/request	LLM calls in wave divided by requests in wave.
Marginal tokens/request	Total LLM tokens in wave divided by requests in wave.
Marginal provider cost/request	Measured provider cost in wave divided by requests in wave.
Cumulative provider cost/request	Running provider cost divided by requests processed.

The central publication figure should plot marginal provider cost per request by wave for all three architectures. A genuine compounding effect requires a defensible downward trend caused by validated reuse, not merely a lower final total.

11. Primary Hypotheses

H1 - Persistence hypothesis. If validated knowledge compounds, the Compounding Smart Agent should require fewer LLM calls per request in later compatible waves than in its initial learning wave.

H2 - Persistence-vs-episodic hypothesis. If cross-wave memory has value, Compounding should avoid reacquisition calls that Episodic Reuse must pay again after each reset.

H3 - Correctness hypothesis. Cost reduction is acceptable only if deterministic reuse does not create stale-reuse errors or materially reduce correctness relative to Stateless Reason.

H4 - Drift-resilience hypothesis. Contract drift should cause selective invalidation/relearning only for affected capabilities; stable capabilities should remain reusable.

H5 - Novelty hypothesis. Late-arriving new families must still trigger fresh reasoning rather than being forced through an unrelated learned capability.

H6 - Cost-curve hypothesis. Any claim that the agent “gets cheaper” must be supported by per-wave marginal measurements, not inferred from architecture design.

These are hypotheses, not expected results. Failure to support one of them is a valid experimental outcome and must be reported without tuning the workload or policy after the canonical run.

12. Success Gates

Gate	Requirement before publication claim
G1	All architectures execute the identical frozen 240-request workload.
G2	Canonical workload hash matches the frozen pre-run hash.
G3	Real provider boundary cannot access hidden ground truth.
G4	Compatibility/promotion/invalidation decisions are deterministic and auditable.
G5	Compounding registry persists across wave boundaries; Episodic registry demonstrably resets.
G6	Per-wave calls, tokens, provider cost, reuse, acquisition, relearning, and errors are recorded.
G7	Independent audit recomputes correctness and summary metrics from request ledgers.
G8	Audit checks for stale reuse and invalid deterministic execution after drift.
G9	Audit verifies that no future-workload fields appear in routing/promotion evidence.
G10	Frozen evidence package includes manifests, ledgers, workload, hashes, audit result, and environment metadata.

13. Provider and Runtime Safety Boundary

As in Chapter 11, the benchmark must not execute arbitrary model-generated Python, shell commands, SQL, network calls, or dynamic code. The model may select or parameterize operations from a closed, declared operation set. Trusted deterministic software executes those operations.

```
LLM output (structured)
-> schema validation
-> operation allowlist
-> typed semantic validation
-> candidate capability
-> validation suite
-> capability registry
-> trusted deterministic runtime
```

This keeps the experiment focused on the economics of accumulated reusable reasoning rather than on arbitrary code-generation risk. The publication must describe the artifact accurately as constrained executable capability or validated operation selection, not unrestricted self-modifying code.

14. Real-Provider Protocol

The implementation should first pass deterministic mock tests, methodology tests, anti-leakage checks, and evidence-audit tests. A small real-provider demo may then verify the public contract, structured output, token accounting, and provider boundary.

The canonical real benchmark must not run until all gates are clean. Once it runs successfully and the independent audit passes, the evidence is frozen and must not be rerun merely to obtain more favorable economics.

The real-provider manifest should include at minimum: run ID, mode, provider, model, Python version, platform, request count, canonical workload hash, pricing constants, policy version/hash, capability-contract version/hash, operation-registry hash, and benchmark code/evidence version.

15. Evidence Package

The frozen canonical evidence ZIP should contain enough information for a third party to reconstruct the chapter tables without trusting the prose.

Artifact	Required content
manifest.json	Environment, provider/model, hashes, policy and contract provenance.
workload.jsonl	All 240 ordered public benchmark requests.
ground_truth.jsonl	Scoring-only expected results; never provider-visible.
stateless_reason_ledger.jsonl	Per-request route, answer, usage, cost, correctness.
episodic_reuse_ledger.jsonl	Per-request route plus reset/acquisition/reuse events.
compounding_ledger.jsonl	Per-request route plus registry/capability provenance.
capability_events.jsonl	PROMOTE, REUSE, INVALIDATE, RELEARN, RETIRE events.
wave_summary.json	Calls/tokens/cost/correctness/reuse by wave and architecture.
summary.json	Whole-run canonical totals.
audit.json	Independent recomputation and PASS/FAIL errors.
README.txt	Exact commands, evidence meaning, and hash instructions.

The final ZIP SHA-256 and byte size become part of Chapter 13. After freezing, the evidence package is immutable.

16. Publication Figures Planned

Figure	Purpose
13.1 - The Compounding Cost Curve	Conceptual opening: reasoning cost falls as validated capability accumulates, with occasional upward steps for novelty/drift.
13.2 - Stateless vs. Episodic vs. Compounding	Main architecture comparison showing registry persistence/reset behavior.
13.3 - Four Benchmark Waves	Timeline of Learn, Reuse, Drift, Mature + Novel.
13.4 - Capability Lifecycle	Unknown -> reason -> validate -> promote -> reuse -> invalidate -> relearn/retire.
13.5 - Marginal LLM Calls by Wave	Canonical measured calls/request for all architectures.
13.6 - Marginal Tokens and Provider Cost by Wave	Canonical measured cost curve; no estimates substituted for measurements.
13.7 - Capability Registry Growth and Repair	Promotions, active capabilities, invalidations, relearning across waves.
13.8 - Where the Remaining LLM Calls Went	Breakdown of acquisition, novelty, reasoning-required exceptions, and relearning.

17. Failure Conditions and Interpretation Rules

The experiment is considered methodologically failed - and must be fixed before canonical publication - if any architecture receives a different workload, if hidden ground truth reaches the provider, if the Compounding architecture sees future drift, if registry persistence/reset behavior is not auditable, or if the independent audit cannot reconstruct the reported metrics.

The experimental thesis is not considered failed merely because the Compounding architecture does not win on cost. A flat or rising marginal cost curve would be important evidence that maintenance, novelty, or insufficient recurrence prevented learning from amortizing. That result must be reported rather than tuned away.

Likewise, if Episodic Reuse performs nearly as well as Compounding, the correct conclusion would be that cross-wave persistence added little value under this workload. If Stateless Reason remains cheapest, the correct conclusion would be that the reusable-capability machinery did not recover its cost.

NO RESULT SHOPPING

After the canonical real benchmark begins, do not change workload composition, drift locations, promotion rules, contract semantics, or cost accounting to make the Compounding architecture win. Any material methodological correction requires a versioned specification amendment and a clearly identified new benchmark version.

18. Relationship to Chapters 11, 12, and 14

Chapter 11 established executable memory: validated reasoning can become reusable capability. Chapter 12 established the economic boundary: reusable capability should be treated as an investment rather than an automatic virtue. Chapter 13 now asks whether those capabilities can accumulate into a system whose marginal reasoning cost falls over time.

Chapter 14 will address proof and measurement discipline across the broader book. Chapter 13 therefore must produce evidence that is easy to audit: explicit per-wave cost curves, route attribution, capability provenance, and immutable frozen artifacts.

19. Frozen Pre-Implementation Decisions

Item	v1.0 decision
Chapter title	Build Agents That Get Cheaper
Subtitle	Designing Compounding Agent Systems Whose Marginal Reasoning Cost Falls as Validated Knowledge Accumulates
Scenario	HarborPoint Distribution Group
Architectures	Stateless Reason / Episodic Reuse / Compounding Smart Agent
Canonical requests	240 per architecture
Waves	4 waves x 60 requests
Operational families	8
Persistence rule	Only Compounding carries validated capabilities across waves
Novelty	At least two families introduced after Wave 1
Drift	Selected previously learned families change in Wave 3
Ground truth	Scoring/audit only; unavailable to provider and routing
Execution safety	Closed operation set; no arbitrary generated code execution
Primary economic view	Per-wave marginal LLM calls, tokens, and measured provider cost
Canonical evidence	Immutable after successful audit/freeze

Exact workload interleaving, contract values, policy thresholds, model/provider configuration, and pricing constants are intentionally not invented in this specification. They must be implemented, tested, documented, and frozen before the canonical run. If any of those choices materially changes the experimental meaning described here, publish a versioned specification amendment before collecting canonical evidence.

FROZEN PRINCIPLE

Reason for what is new. Validate what can become reusable. Carry forward only what remains compatible. Repair what changes. A Smart Agent gets cheaper only when validated knowledge accumulates faster than the cost of maintaining it.

END OF EXPERIMENTAL SPECIFICATION v1.0